

CS-200 — Partie 2b : Processeur, I/O et exceptions (notes de cours)

Oct 9, 2026 · @Gab

Ces notes suivent les slides 2 à 25 du cours CS-200 (Paolo Ienne) : comment le processeur communique avec la mémoire et les périphériques, du décodage d'adresse jusqu'aux limites du polling. Elles signalent aussi les endroits où les slides simplifient ou contiennent une erreur, notamment un piège `lui / addi` à la slide 19 qui revient souvent en labo.

1. Le CPU vu de l'extérieur (slides 2–3)

Vu de l'extérieur, le CPU est un gros circuit séquentiel qui expose une **interface mémoire**. Cette partie du cours ne regarde plus l'intérieur du processeur (contrôle et datapath) mais ses échanges avec la mémoire et les périphériques, les trois autres composants classiques d'un ordinateur.

L'interface mémoire de la slide 3 comporte quatre groupes de signaux :

- `addr` (≥ 32 bits) : l'adresse de l'accès. Le « $\geq$ » vient du fait que l'adresse *physique* peut être plus large que les registres. Par exemple, RV32 avec la pagination Sv32 produit des adresses physiques de 34 bits.
- `wdata` (32 bits) : la donnée à écrire lors d'un store.
- `rdata` (32 bits) : la donnée lue lors d'un load.
- `ctrl` : les signaux de contrôle. **CE** (Circuit Enable) indique qu'un accès a lieu et que l'adresse est valide. **WE** (Write Enable) indique que l'accès est une écriture.

La variante à droite de la slide utilise un **seul bus de données bidirectionnel** au lieu de `rdata` et `wdata` séparés. Le même fil est alors piloté tantôt par le CPU (écriture), tantôt par la mémoire (lecture). Deux sorties ne doivent jamais piloter le même fil en même temps, d'où le besoin de **buffers tristate** (section 6).

2. Carte mémoire et décodage d'adresse (slides 4–5)

Un composant n'est activé que si les bits de poids fort de l'adresse correspondent à sa plage : c'est le **décodage d'adresse**. La carte mémoire physique dit quelle plage de l'espace 32 bits (`0x00000000` – `0xFFFFFFFF`, soit 4 GiB) revient à quel composant.

Sur la slide 4, la RAM se trouve dans la région `0x80000000` – `0x9FFFFFFF`. Le programme y place son code, ses données et sa pile, qui est en haut et croît vers le bas.

Cette région correspond exactement aux adresses dont les 3 bits de poids fort valent 100. En effet, $0x8 = 1000_2$ et $0x9 = 1001_2$, alors que $0xA = 1010_2$ a déjà un bit 29 différent. Une condition « $\geq$ et $<$ » se réduit donc à une égalité sur 3 bits, très bon marché en matériel.

Exemple de la slide 5

L'adresse `0x800A78FA` est coupée en deux champs :

```
addr[31:20] = 0x800    → comparé à la constante 0x800 ("est-ce la RAM ?")
addr[19:0]   = 0xA78FA → envoyé tel quel à la RAM (adresse interne)
```

Le signal d'activation de la RAM vaut :

```
en_RAM = ce ∧ (addr[31:20] == 0x800)
```

Le comparateur d'égalité sur 12 bits est un AND de 12 XNOR bit à bit. Avec 20 bits d'adresse interne, la RAM contient 2^{20} emplacements et occupe la plage `0x80000000` – `0x800FFFFF`. Le `we` passe directement du CPU à la RAM, car si `en = 0` la RAM ignore de toute façon l'accès.

Ce que la slide simplifie

- **Granularité de la RAM.** Si la RAM est organisée en mots de 32 bits, on n'enverrait que `addr[19:2]` et les bits `[1:0]` choisiraient l'octet. Le schéma suppose une RAM adressée à l'octet.
- **Décodage partiel.** Si le comparateur ne testait qu'une partie des bits de poids fort, la même RAM répondrait à plusieurs plages d'adresses (*aliasing*). Ici le décodage est complet : les 12 bits sont comparés.

3. Les périphériques et leurs débits (slides 6–7)

Les débits des périphériques s'étalent sur environ **7 ordres de grandeur**, d'un clavier (~kbps) à l'HDMI 2.1 ou l'USB4 (dizaines de Gbps). Une seule méthode d'accès ne peut donc pas convenir à tous.

Pour un clavier, le CPU peut se permettre de venir voir de temps en temps. Pour un SSD NVMe, il ne pourrait pas déplacer lui-même chaque octet. Cela motive les mécanismes vus ensuite : le polling ici, puis les interruptions et le DMA.

Attention aux unités du tableau : la plupart sont en **bits/s** (kbps, Gbps), mais les disques optiques sont en **octets/s** (KB/s, MB/s), avec un facteur 8 entre les deux.

4. Deux façons d'accéder aux I/O : PMIO vs MMIO (slides 8–10)

Avec le MMIO, les registres des périphériques sont de simples adresses mémoire, accessibles par `lw` et `sw` sans aucun matériel ni instruction spéciale. C'est la solution dominante aujourd'hui, et celle du Lab B.

	PMIO (slide 8)	MMIO (slide 9)
Espace d'adressage	Séparé (numéros de port, 8–16 bits)	Partagé avec la mémoire
Signaux	<code>ctrl-I/O</code> dédiés (CE, OE)	Les mêmes que la mémoire
Instructions	Spéciales : <code>IN reg</code> , <code>port</code> , <code>OUT port</code> , <code>reg</code> (x86)	Les loads et stores ordinaires
Usage actuel	Rare, surtout historique sur x86 (65 536 ports)	Dominant

La carte MMIO de la slide 9

```
0x10000000      LEDs
0x10000010-0x1000002F Display  (0x20 = 32 octets = 8 mots)
0x10000030      Buttons
```

```
lui t0, 0x10000      # t0 = 0x10000 << 12 = 0x10000000
sw  t1, 0(t0)        # écrit t1 dans le registre des LEDs
lw  t2, 0x30(t0)     # lit l'état des boutons (0x10000030)
```

Un accès MMIO n'est **pas** un accès mémoire ordinaire, car il a des *effets de bord*. Lire deux fois les boutons peut donner deux valeurs différentes, et écrire peut déclencher une action. Ces plages ne doivent donc pas passer par le cache, et en C les pointeurs vers des registres MMIO doivent être `volatile`.

Le décodage correspondant (slide 10)

Chaque composant est activé par un comparateur sur l'adresse, combiné avec le type d'accès :

```
en_Mem      = (0x80000000 ≤ addr < 0xA0000000)      = (addr[31:29] == 0b100)
load_LEDs   = (addr == 0x10000000) ∧ write
oe_Buttons  = (addr == 0x10000030) ∧ ¬write
```

- **LEDs.** C'est un registre (basculés D avec enable) : D est relié au bus de données, Q pilote les LEDs. Il ne capture la donnée que lors d'une écriture à la bonne adresse, puis la maintient. Les LEDs restent donc allumées entre deux `sw`.
- **Boutons.** Leur valeur n'est placée sur le bus que lors d'une lecture à la bonne adresse, via un tristate. Le reste du temps, ils ne pilotent pas le bus.

5. Exemple complet : un convertisseur A/D sur un bus asynchrone (slides 11-19)

Cet exemple réunit tout : décodage d'adresse, tristates, handshake `/DTACK` et polling logiciel. Le code de la slide 19 contient en plus une erreur de construction d'adresse à connaître.

Le composant (slide 11)

Convention : un `/` devant un nom (ou une bulle sur le schéma) signifie **actif bas**. `/DV = 0` veut donc dire « donnée valide ».

Signal	Direction	Rôle
<code>START</code>	Entrée	Une impulsion lance une conversion
<code>/DV</code>	Sortie	Passe à 0 quand la conversion est terminée
<code>D7-D0</code>	Sortie (8 bits)	Résultat de la dernière conversion

Sur le chronogramme, une impulsion sur `START` déclenche la conversion. Après le temps de conversion, `/DV` descend et `D7-D0` porte le résultat. Au `START` suivant, `/DV` remonte, car la donnée n'est plus garantie valide.

Le bus (slide 12)

Le bus ressemble à celui du MC68000 : 24 bits d'adresse (16 MiB), 8 bits de données bidirectionnels, et trois signaux de contrôle.

- `/AS` (Address Strobe) : vaut 0 quand une adresse valide est sur le bus.
- `R/W` : vaut 1 pour une lecture, 0 pour une écriture.
- `/DTACK` (Data Acknowledge) : **entrée** du CPU, pilotée par le périphérique.

Le cahier des charges (slide 13)

```
0xFFFFF0 : tout accès (R ou W) lance une conversion
0xFFFFF4 : lecture → bit 0 = DV
0xFFFFF8 : lecture → résultat de la conversion
```

Le rôle de /DTACK (slides 14–16)

C'est le périphérique, et non le CPU, qui décide de la durée de l'accès. Le CPU suit une petite machine à états :

1. Il place l'adresse sur le bus et met $\text{/AS} \leftarrow 0$.
2. Il reste dans cet état tant que $\text{/DTACK} == 1$. Chaque cycle passé à attendre est un *wait state*.
3. Quand $\text{/DTACK} == 0$, il considère l'accès comme terminé : la donnée lue est prête, ou la donnée écrite a été capturée.

C'est un bus **asynchrone** : un composant lent peut faire attendre le CPU sans que celui-ci connaisse sa latence à l'avance. Avec l'interface synchrone de la slide 3, au contraire, la latence est fixée par le protocole.

Le revers : si personne ne répond (adresse non décodée), le CPU attend indéfiniment. Le vrai 68000 a pour cela un signal /BERR (bus error) déclenché par un timeout externe, ce qui fait le lien avec la partie « Exceptions » du chapitre.

Le circuit (slide 17)

On note $\text{sel}_X = (\text{addr} == X)$ la sortie de chaque comparateur et $\text{AS} = \neg \text{/AS}$ (accès en cours). Chaque porte du schéma s'écrit alors :

```
START  = AS ∧ sel_FFFFF0          (indépendant de R//W → "tout accès")
en_DV  = AS ∧ sel_FFFFF4 ∧ (R//W = 1)  (lecture uniquement)
en_D   = AS ∧ sel_FFFFF8 ∧ (R//W = 1)  (lecture uniquement)

/DTACK = ¬(START v en_DV v en_D v ...)  (porte NOR)
```

- **AND à entrée inversée.** La bulle sur l'entrée reliée à /AS réalise $\neg \text{/AS}$. La porte en bas à gauche n'a pas d'entrée R//W , ce qui implémente « tout accès, lecture ou écriture ».
- **Tristate de /DV .** Il est inverseur (bulle en sortie) et place donc $\text{DV} = \neg \text{/DV}$ sur le bus : **1 quand la donnée est valide**. C'est cohérent avec le logiciel, qui boucle tant qu'il lit 0.
- **Tristate de D7-D0 .** Il place les 8 bits du résultat sur le bus quand $\text{en}_D = 1$.

- **NOR pour /DTACK**. Dès qu'un des trois accès est sélectionné, /DTACK passe à 0 immédiatement, sans wait state. C'est possible parce que les « registres » du convertisseur sont des fils déjà stables. Les ... indiquent que d'autres périphériques ajoutent leurs propres acquittements sur la même porte.

Le logiciel (slides 18-19)

```
read_adc: lui    t1, 0xffff
          addi   t1, t1, 0xff0    # t1 = 0xfffff0 (?)
          sw     zero, 0(t1)      # écriture en F0 → START
poll:     lw     t0, 4(t1)        # lit DV
          beqz   t0, poll         # boucle tant que DV == 0
end:      lw     a0, 8(t1)        # lit le résultat
          ret
```

On lance la conversion par une écriture factice (la valeur `zero` n'a aucune importance, seul l'accès compte). On attend ensuite en lisant DV en boucle, c'est du **polling**, puis on lit le résultat.

Erreur dans la slide 19 : l'extension de signe de `addi`

L'immédiat de `addi` fait 12 bits **signés**, donc `0xff0` vaut -16 et non $+4080$. Selon l'assembleur, la ligne est soit refusée (4080 sort de l'intervalle $[-2048, 2047]$), soit interprétée comme -16 :

```
lui    t1, 0xffff    → t1 = 0x00FFF000
addi   t1, t1, -16    → t1 = 0x00FFEFF0    ≠ 0x00FFFFF0
```

Pour construire correctement la constante, on compense l'extension de signe en ajoutant 1 à la partie haute :

```
lui    t1, 0x1000    # t1 = 0x01000000
addi   t1, t1, -16    # t1 = 0x00FFFFF0
```

Plus simplement, `li t1, 0xFFFFF0` laisse l'assembleur faire ce calcul. Règle générale : si le bit 11 de la partie basse vaut 1, la partie passée à `lui` doit être $(\text{valeur} \gg 12) + 1$.

Deux autres points de rigueur

- **Masquage de DV.** `beqz t0` teste les 32 bits, alors que le circuit ne pilote que le bit 0. Si les autres lignes du bus ne sont pas garanties à 0, il faut un `andi t0, t0, 1` avant le test. L'exemple UART de la section 7, lui, fait ce masquage.

- **Hypothèse temporelle implicite.** Le code suppose que `/DV` remonte *immédiatement* au START. Sinon, le premier `lw` du polling pourrait voir le DV de la conversion précédente et renvoyer l'ancien résultat. C'est une propriété du composant à vérifier dans sa datasheet.

6. Les buffers tristate et le système du Lab B (slides 20–21)

Deux buffers tristate dont les sorties sont reliées forment un **multiplexeur décentralisé**. Dans le Lab B, sur FPGA, cette fonction est réalisée par un vrai mux.

Fonction logique

```
enable = 1 → sortie = entrée (0 ou 1)
enable = 0 → sortie = Z (haute impédance : le buffer est "débranché" du fil)
```

Avec A activé par `¬S` et B activé par `S`, sorties reliées :

```
C = S ? B : A
```

C'est exactement un multiplexeur 2→1, mais chaque source se connecte elle-même au bus quand elle est sélectionnée, sans composant central. On peut donc ajouter des périphériques sur un bus partagé sans modifier le reste.

Les deux dangers

- **Deux buffers actifs avec des valeurs différentes.** C'est un court-circuit : un transistor tire vers VDD, l'autre vers GND. Il en résulte un fort courant, un niveau logique indéfini et un échauffement. Dans le circuit A/D, c'est le décodage d'adresse qui garantit qu'au plus un tristate est actif à la fois.
- **Aucun buffer actif.** Le bus flotte et sa valeur est indéfinie, d'où la remarque sur le masquage de DV en section 5.

Le système du Lab B (slide 21)

Un FPGA n'a généralement pas de tristates *internes*, donc on utilise un mux, qui a exactement la même fonction.

- Le **décodeur** prend `addr` et produit `en_ram`, `en_leds`, `en_7_seg_lcd` et `en_buttons`, comme les comparateurs des slides 5 et 10.
- Tous les périphériques reçoivent en parallèle `addr`, `wdata` et `we`. Seul celui dont `en = 1` réagit.

- Côté lecture, seuls la RAM et les boutons ont un `rdata`. Les LEDs et l'afficheur 7 segments sont en écriture seule. Le mux choisit `rdata = en_buttons ? buttons.rdata : ram.rdata`.

Conséquence directe : lire à l'adresse des LEDs renvoie la sortie de la RAM, donc une valeur sans signification. Un périphérique en écriture seule ne peut pas être relu. Si un programme a besoin de savoir ce qu'il a affiché, il doit en garder une copie en RAM.

7. Périphériques programmables : l'exemple de l'UART (slides 22-24)

Un périphérique élaboré expose un **ensemble de registres** à des adresses distinctes : données en entrée et en sortie, commande et statut. Le convertisseur A/D suivait déjà ce modèle, avec F0 pour la commande, F4 pour le statut et F8 pour la donnée.

Les registres de l'UART (slide 23)

Adresse	Registre	Contenu
0x10000000	CTRL	Bit 7 = enable, bits 2..0 = vitesse (0b001 = 9600 baud)
0x10000004	STATUS	Bit 1 = donnée reçue disponible, bit 0 = prêt à émettre
0x10000008	DATAIN	Octet reçu
0x1000000C	DATAOUT	Octet à émettre

Le code de send_string (slide 24)

1. **Configuration.** `t4 = 0x80 | 0x01 = 0x81`, écrit dans CTRL, active l'UART à 9600 baud.
2. **Boucle sur les caractères.** `lb t5, 0(a0)` charge l'octet courant de la chaîne. On s'arrête sur le `\0` terminal.
3. **Attente puis émission.** On lit STATUS, on **masque** le bit 0 avec `andi` et on boucle tant qu'il vaut 0. Quand l'UART est prête, on écrit l'octet dans DATAOUT, on avance le pointeur et on recommence.

Remarques

- Les adresses sont chargées avec `li`. L'assembleur gère donc correctement le découpage `lui / addi`, contrairement à la slide 19.

- Les commentaires de la slide ont deux incohérences mineures. `UART_CTRL_ADDR` est commenté « status register », et la vitesse est décrite sur 4 bits `[3:0]` alors que la slide 23 dit bits 2..0. Se fier à la slide 23.
- Reconfigurer l'UART à chaque appel est inoffensif mais inhabituel. En pratique, l'initialisation se fait une seule fois au démarrage.
- La fonction n'utilise que des registres `t*` et `a0`, tous *caller-saved*. Elle n'a besoin ni de pile ni de sauvegarde : c'est une fonction feuille propre.

8. Les limites du polling (slide 25)

Le polling gaspille plus de 99,9 % du temps CPU face à un périphérique lent, et risque de perdre des données face à un périphérique rapide. C'est ce qui motive les interruptions et le DMA.

Le coût, chiffré

À 9600 baud avec un format 8N1 (1 bit de start, 8 bits de données, 1 bit de stop), chaque caractère fait 10 bits. L'UART émet donc 960 caractères par seconde, soit un caractère toutes les $\approx 1,04$ ms. Un CPU à 100 MHz exécute pendant ce temps :

$$1,04 \cdot 10^{-3} \text{ s} \times 10^8 \text{ cycles/s} \approx 104\,000 \text{ cycles par caractère}$$

Presque tous ces cycles sont passés dans la boucle `check_tx_ready` à ne rien faire d'utile.

Le problème inverse

En réception, si l'UART ne peut stocker qu'un seul octet reçu, le CPU doit venir le lire avant l'arrivée du suivant, sinon la donnée est écrasée. Le polling impose donc un compromis sans bonne solution : interroger souvent gaspille du temps, interroger rarement fait perdre des données. Avec plusieurs périphériques, il faut en plus les visiter tous à tour de rôle.

La suite du chapitre

Avec les **interruptions**, c'est le périphérique qui signale au CPU qu'il a besoin de lui, et le CPU fait autre chose en attendant. Pour les périphériques très rapides, le **DMA** transfère en plus les données sans passer par le CPU.